PLAN DE PRUEBAS

1. Supuestos previos para las pruebas

Antes de ejecutar los casos:

- Backend levantado (`cd backend && npm start`).

- Frontend levantado (`cd frontend && npm start`).

- Base de datos con:

  - Varios talleres creados.

- Navegador limpio o al menos:

  - Sin sesiones anteriores (puedes limpiar `localStorage` o usar ventana de incógnito).

2. Pruebas de registro y login

2.1. Registro de tutor

Objetivo: comprobar que el alta de usuario desde la web funciona y valida bien.

- *Caso R1 – Registro correcto*

  - Pasos:

    1. Ir a `http://localhost:3000`.

    2. Pulsar en “Registrarme”.

    3. Rellenar nombre, apellidos, email no usado, teléfono opcional y contraseña (>= 6 chars).

    4. Enviar.

  - Resultado esperado:

    - Mensaje de éxito: usuario registrado.

    - No hay errores en consola.

    - En BD aparece un nuevo registro en `tutores` con `password_hash` (no en texto plano).

- *Caso R2 – Email duplicado*

- Pasos:

  1. Repetir el registro con el mismo email del caso R1.

- Resultado esperado:

  - Mensaje de error tipo “El email ya está registrado”.

  - No se crea un nuevo tutor en BD.

- *Caso R3 – Contraseña demasiado corta*

- Pasos:

  1. Intentar registrar con contraseña de menos de 6 caracteres.

- Resultado esperado:

  - El formulario no deja avanzar o devuelve error adecuado (del lado cliente/servidor).

  - No se crea tutor.

<u>2.2. Login</u>

Objetivo: validar que solo usuarios correctos entran y que se guarda el token.

- *Caso L1 – Login correcto*

- Pasos:

  1. Usar el email/contraseña del caso R1.

- Resultado esperado:

  - Se entra al panel (se ve “Bienvenido, {nombre}”).

  - En `localStorage` hay un `token`.

  - No hay mensajes de error.

- *Caso L2 – Login con contraseña incorrecta*
  - Pasos:
    1. Poner email correcto + contraseña incorrecta.

  - Resultado esperado:
    - Mensaje “Usuario o contraseña incorrectos”.
    - No se entra al panel.

- *Caso L3 – Logout*
  - Pasos:
    1. Estando en el panel, pulsar “Cerrar sesión”.

  - Resultado esperado:
    - Vuelve a la pantalla de Login.
    - Se borra el token de `localStorage`.

3. Pruebas de gestión de talleres y plazas

Objetivo: asegurar que los talleres se muestran bien y que el contador de plazas funciona.

- *Caso T1 – Visualización de talleres*
  - Pasos:
    1. Con un tutor logueado, abrir la sección “Nueva inscripción”.

  - Resultado esperado:
    - En el select de “Taller” aparecen varios talleres en formato:
    - `Nombre del taller (X/Y plazas)`, con X = ocupadas, Y = cupo.

- *Caso T2 – Actualización de plazas ocupadas tras nueva inscripción*
  - Pasos:
    1. Anotar valor actual en un taller, por ejemplo `0/20`.
    2. Crear una nueva inscripción en ese taller (ver apartado siguiente).
    3. Volver a desplegar la lista de talleres.

  - Resultado esperado:
    - El mismo taller ahora muestra `1/20` (u `X+1/Y`).
    - No es necesario recargar la página ni relogear.

4. Pruebas de inscripciones

4.1. Creación básica

- *Caso I1 – Inscripción correcta*
  - Pasos:
    1. Estando logueado, en “Nueva inscripción”, rellenar:
       - Nombre y apellidos del participante.
       - Fecha de nacimiento válida (pasada, no futura).
       - Elegir un taller con plazas libres.
    2. Pulsar “Inscribir”.

  - Resultado esperado:

    - Mensaje `¡Inscripción realizada correctamente!`.
    - El formulario se limpia.
    - La nueva inscripción aparece en la tabla “Mis inscripciones”.
    - El contador de plazas del taller se incrementa.

- *Caso I2 – Fecha de nacimiento futura*

  - Pasos:

    1. Intentar inscribir con una fecha de nacimiento posterior a hoy.

  - Resultado esperado:

    - El formulario muestra “La fecha de nacimiento no puede ser posterior a hoy.”.

    - El backend, si se llegase a llamar directamente, también rechaza la petición.

    - No se crea inscripción en BD.

- *Caso I3 – Cupo máximo alcanzado*

  - Pasos:

    1. Elegir un taller con cupo pequeño.

    2. Inscribir participantes hasta llegar al cupo.

    3. Intentar una inscripción adicional al mismo taller.

  - Resultado esperado:

    - El backend devuelve un error tipo “No quedan plazas disponibles en este taller”.

    - La UI lo muestra en rojo.

    - La tabla “Mis inscripciones” no añade una nueva entrada.

    - El contador de plazas no sobrepasa el cupo.

- *Caso I4 – Duplicidad de participante en el mismo taller*

  - Pasos:

    1. Inscribir un participante “Nombre Apellido” en taller X.

2. Intentar inscribir de nuevo el mismo “Nombre Apellido” en el mismo taller X (mismo tutor).

- Resultado esperado:
  - Mensaje de error tipo “Participante ya inscrito en este taller”.
  - No aparece una segunda línea duplicada en la tabla.

5. Pruebas de “Mis inscripciones”: cancelar y pagar

5.1. Cancelar

- *Caso C1 – Cancelar inscripción pendiente*
  - Pasos:
    1. En la tabla “Mis inscripciones”, localizar una inscripción en estado `pendiente`.
    2. Pulsar “Cancelar”.
    3. Confirmar en el diálogo.
  - Resultado esperado:
    - El estado de esa fila pasa a `Cancelada` (badge gris).
    - No aparece botón “Pagar”.
    - Si vuelves a mirar el contador del taller, `ocupadas` debe haber bajado en 1.

5.2. Pagar

- *Caso P1 – Marcar como pagada una inscripción pendiente*

- Pasos:

  1. En la tabla, localizar una inscripción `pendiente`.

  2. Pulsar “Pagar”.

- Resultado esperado:

  - El estado pasa a `Pagado` (badge verde).

  - En la tabla de `pagos` debe aparecer un nuevo pago con estado `realizado` y método `simulado`.

  - Si cancelas/deshaces, la UI y BD mantienen coherencia (no se permite pagar canceladas).

- *Caso P2 – No permitir pagar canceladas*

- Pasos:

  1. Cancelar una inscripción.

  2. Verificar que no aparece botón “Pagar”.

- Resultado esperado:

  - El frontend no ofrece la acción “Pagar” en inscripciones canceladas.

  - El backend, si se llama forzando la URL, debería igualmente validar el estado (opcionalmente).

6. Pruebas de seguridad básica (JWT)

- *Caso S1 – Acceso a `/api/inscripciones/tutor/:tutorId` sin token*

- Pasos:

  1. Hacer una petición a esta ruta sin cabecera Authorization (por ejemplo con un cliente REST o desde devtools).

- Resultado esperado:

- Respuesta 401 con mensaje tipo “No autenticado”.

- *Caso S2 – Token inválido o expirado*

- Pasos:

1. Modificar el token en `localStorage` o usar un token inválido en la cabecera.

- Resultado esperado:

- Respuesta 401 “Token inválido o expirado”.

- La UI debería gestionar el caso (por ejemplo, forzando logout en siguientes mejoras).

7. Pruebas de reportes y descarga de ficheros

7.1. Reporte de inscripciones

- *Caso RPT1 – Descargar CSV de inscripciones de un taller*

- Pasos:

1. Tener varias inscripciones en un mismo taller (con diferentes estados).
2. En el navegador, ir a:
   - `http://localhost:3000/api/reportes/inscripciones/ID_TALLER`

- Resultado esperado:

- Se descarga un fichero `reporte_inscripciones_taller_ID_TALLER.csv`.

- Al abrirlo:

  - Aparecen columnas esperadas (id_inscripcion, participante, taller, tutor, estado…).

  - El número de filas coincide con las inscripciones existentes en ese taller.

7.2. Reporte de asistencia

- *Caso RPT2 – Descargar CSV de asistencia*

- Pasos:

1. Asegurarse de tener registros en `asistencia`.
2. Ir a:
   - `http://localhost:3000/api/reportes/asistencia/ID_TALLER`

- Resultado esperado:

- Se descarga `reporte_asistencia_taller_ID_TALLER.csv`.
- Filas y columnas (fecha, presente, comentarios, participante…) coinciden con la BD.

7.3. Reporte de pagos

- *Caso RPT3 – Descargar CSV de pagos*

- Pasos:

1. Haber marcado varias inscripciones como pagadas.
2. Ir a:
   - `http://localhost:3000/api/reportes/pagos`

- Resultado esperado:

- Se descarga `reporte_pagos.csv`.
- Cada fila corresponde a un pago registrado.
- Campos como `monto`, `status`, `taller`, `tutor` son coherentes.

8. Pruebas de experiencia de usuario / errores

- *Caso UX1 – Mensajes claros en errores de red*

  - Pasos:

    1. Apagar el backend y, con el frontend encendido, intentar loguear o inscribir.

  - Resultado esperado:

    - Aparecen mensajes tipo “Error al conectar con el servidor”.

    - La app no se rompe (no pantalla en blanco con error JS).

- *Caso UX2 – Validaciones de campos vacíos*

  - Probar envíos de formularios con campos obligatorios vacíos:

    - Registro sin nombre/apellido/email/contraseña.

    - Inscripción sin taller, sin fecha de nacimiento, etc.

  - Resultado esperado:

    - El navegador/React no dejan enviar o muestran errores apropiados.

    - El backend, si recibe algo incompleto, responde con error de validación